3. TAREA

C.I.F.P A CARBALLEIRA

Miguel Ángel Domínguez Llorente

2025-2026

Miguel Angel Dominguez Llorente

Índice

1.-Análisis de caja blanca 6

2- Análisis de caja negra 8

3.Junit5 10

4.Puntos de ruptura 11

Documento breakpoint: ..... **¡Error! Marcador no definido.**

Detalles de la tarea de esta unidad.

Enunciado.

En esta tarea se considera una clase **Java** CCuenta que dispone de los métodos main, ingresar y retirar. Este es el código de los métodos main, ingresar y retir que deberás tener en cuenta para resolver la tarea:

Método main

```
public static void main(String[] args) {
        // Depuracion. Se detiene siempre
        CCuenta miCuenta = new CCuenta();
        System.out.println("Saldo Incial: " + miCuenta.dSaldo + " euros");
        // Depuracion. Provoca parada por ingreso con cantidad menor de 0
        miCuenta.ingresar(-100);
        System.out.println("Saldo Incial: " + miCuenta.dSaldo + " euros");
        miCuenta.ingresar(100);
        System.out.println("Saldo tras ingreso: " + miCuenta.dSaldo + "
euros");
        miCuenta.ingresar(200);
        System.out.println("Saldo tras ingreso: " + miCuenta.dSaldo + "
euros");
        // Depuracion. Provoca parada con codicion de tercer ingreso
        miCuenta.ingresar(300);
        System.out.println("Saldo tras ingreso: " + miCuenta.dSaldo + "
euros");
        miCuenta.retirar(50);
        System.out.println("Saldo tras retirada: " + miCuenta.dSaldo + "
euros");
}
```

Método ingresar

```
public int ingresar(double cantidad)
      int iCodErr;
       if (cantidad < 0)
      {
                System.out.println("No se puede ingresar una cantidad
negativa");
                iCodErr = 1;
       }
       else if (cantidad == -3)
      {
```

```
                System.out.println("Error detectable en pruebas de caja
blanca");
                iCodErr = 2;
        }
        else
       {
               // Depuracion. Punto de parada. Solo en el 3 ingreso
               dSaldo = dSaldo + cantidad;
                iCodErr = 0;
        }
        // Depuracion. Punto de parada cuando la cantidad  es menor de 0
        return iCodErr;
}
```

Método retirar

```
public void retirar (double cantidad)
    {
        if (cantidad <= 0)
        {
            System.out.println("No se puede retirar una cantidad negativa");
        }
        else if (dSaldo < cantidad)
        {
            System.out.println("No se hay suficiente saldo");
        }
        else
        {

        }
    }
```

Deberás realizar un documento donde dar respuesta a los siguientes apartados:

1. Realiza un **análisis de caja blanca** completo del método ingresar.
2. Realiza un **análisis de caja negra**, incluyendo valores límite y conjetura de errores del método retirar. Debes considerar que este método recibe como parámetro la cantidad a retirar, que no podrá ser menor a 0. Además en ningún caso esta cantidad podrá ser mayor al saldo actual. Al tratarse de pruebas funcionales no es necesario conocer los detalles del código pero te lo pasamos para que lo tengas.

3. Crea la clase CCuentaTest del tipo **Caso de prueba JUnit** en **Eclipse** que nos permita pasar las pruebas unitarias de caja blanca del método ingresar. Los casos de prueba ya los habrás obtenido en el primer apartado del ejercicio. Copia el código fuente de esta clase en el documento.
4. Genera los siguientes **puntos de ruptura** para validar el comportamiento del método ingresar en modo depuración.
    - Punto de parada sin condición al crear el objeto miCuenta en la función main. Línea 3 del código del método main que se presenta en la siguiente página de este libro.
    - Punto de parada en la instrucción return del método ingresar sólo si la cantidad a ingresar es menor de 0. Línea 20 del código del método ingresar que se presenta más adelante.
    - Punto de parada en la instrucción donde se actualiza el saldo, sólo deberá parar la tercera vez que sea actualizado. Línea 16 del código del método ingresar que se presenta más adelante.

Pulsando el botón derecho sobre la ventana de puntos de ruptura selecciona la opción "Exportar puntos de ruptura". Seleccionas los tres puntos de ruptura generados y guardas el fichero. El fichero tendrá la extensión bkpt, la cambias por txt. Ahora abres el fichero y copias el contenido íntegramente al documento.

Criterios de puntuación. Total 10 puntos.

Los criterios de puntuación serán los siguientes.

- Pruebas de caja blanca. 2,5 puntos.
- Pruebas de caja negra. 2,5 puntos.
- 2,5 puntos.
- Depuración. 2,5 puntos.

Recursos necesarios para realizar la Tarea.

Ordenador con un **IDE que permita programar en Java. Si es Eclipse mejor, porque el proyecto inicial y la solución está en Eclipse**.

Pulsa aquí para descarga el proyecto **Java "java-sw.rar"**.

Consejos y recomendaciones.

Se pretenden que partiendo de una clase con dos métodos ya diseñados, se utilicen herramientas de depuración para corregir errores en el código. A continuación, se deberá diseñar todos los casos de prueba, para verificar el comportamiento de los métodos ingresar y retirar.

Indicaciones de entrega.

Una vez realizada la tarea elaborarás un único documento donde figuren las respuestas correspondientes. El envío se realizará a través de la plataforma de la forma establecida para ello, y el archivo se nombrará siguiendo las siguientes pautas:

apellido1_apellido2_nombre_SIGxx_Tarea

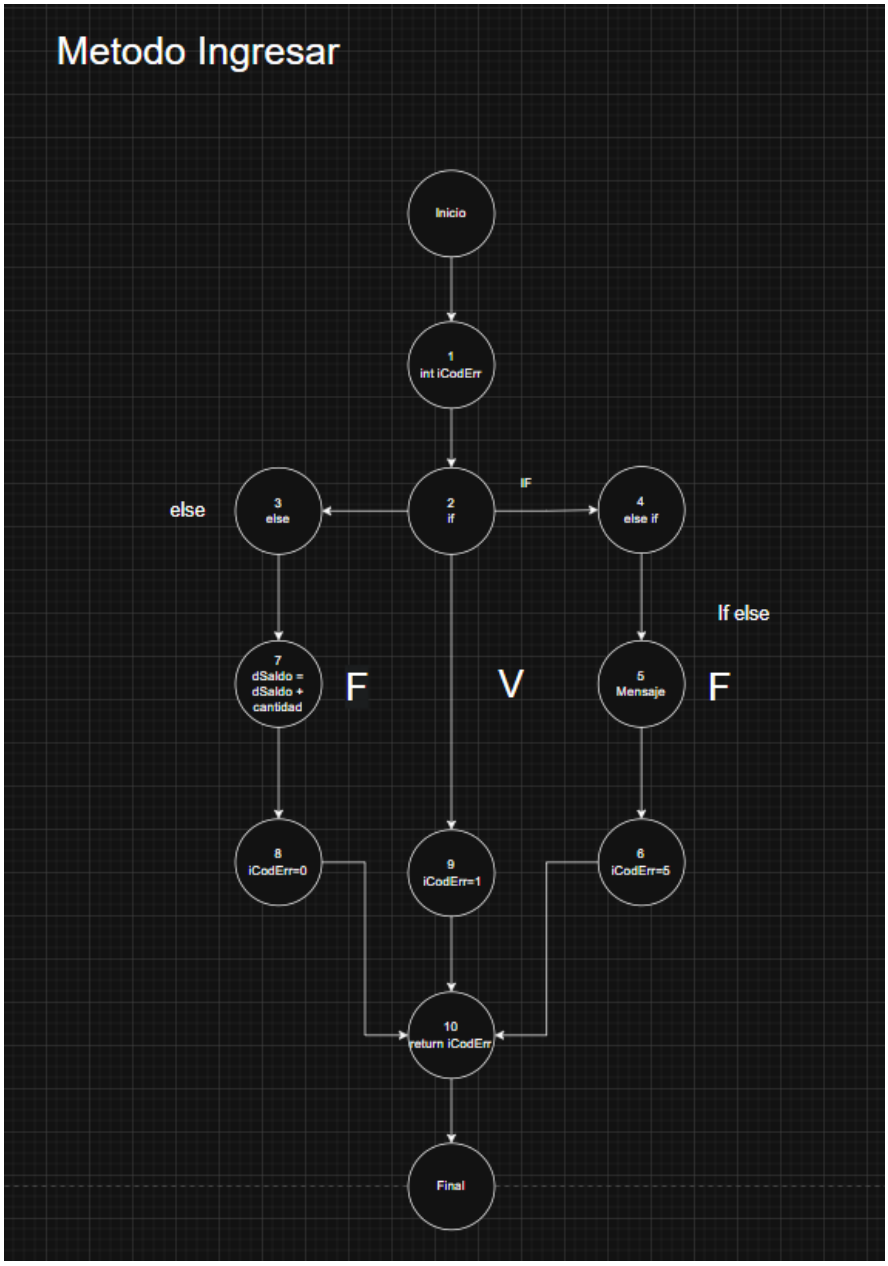
Asegúrate que el nombre no contenga la letra ñ, tildes ni caracteres especiales extraños. Así por ejemplo la alumna **Begoña Sánchez Mañas para la tercera unidad del MP de ED**, debería nombrar esta tarea como...

sanchez_manas_begona_ED03_Tarea

1.-Análisis de caja blanca

A)Diagrama de flujo

B)Grafo de flujo de control



C) complejidad ciclomática (VG)

Método de cálculo	Complejidad	Comentario
Nº de nodos/regiones	10	
Nº de aristas -Nº nodos +2	V(G) = 11 -10 + 2	
Nº de condiciones +1	3+1	

D)Caminos linealmente independientes

Camino 1: 1,2,3,7,8,10
Camino 2: 1,2,4,5,6,10
Camino 3: 1,2,9,10

Hemos visto que podemos tener 3 caminos hasta llegar al return iCodError

F) Casos de prueba

Numero	Camino independiente
1	1,2,3,7,8,10
2	1,2,4,5,6,10
3	1,2,9,10

G) Caso de uso, resultado esperado y análisis

Caminos / Casos de uso	Datos		Salidas	
	cantidad		iCodErr	
1	0…1000		0	
2	-1		1	
3	-3		1	

2- Análisis de caja negra

Pasos a seguir
A) determinar las claves de equivalencia

son valores de entrada

clase de equivalencia 1

Entrada : cantidad<0
comportamiento : no se puede obtener cantidad negativos

clase de equivalencia 2

Entrada dSaldo < cantidad
no puede haber una cantidad menor que dSaldo

clase de equivalencia 3

Entrada cantidad > 0
Comportamiento : no devuelve un error y el valor no está vacío y es positivo

c) Análisis de valores limites

Ejemplo	Clases de equivalencia	Valores límites
En caso de un valor negativo	Cantidad <0	-1
En caso de Cantidad 0	Cantidad =0	0
En caso que sea superior	Cantidad > 1	1

d) Conjetura de errores

Posible errores que puede cometer

Cantidad no puede ser negativo
Cantidad obtiene un valor como -3
Cantidad obtiene valores como 0
Cantidad obtiene valores

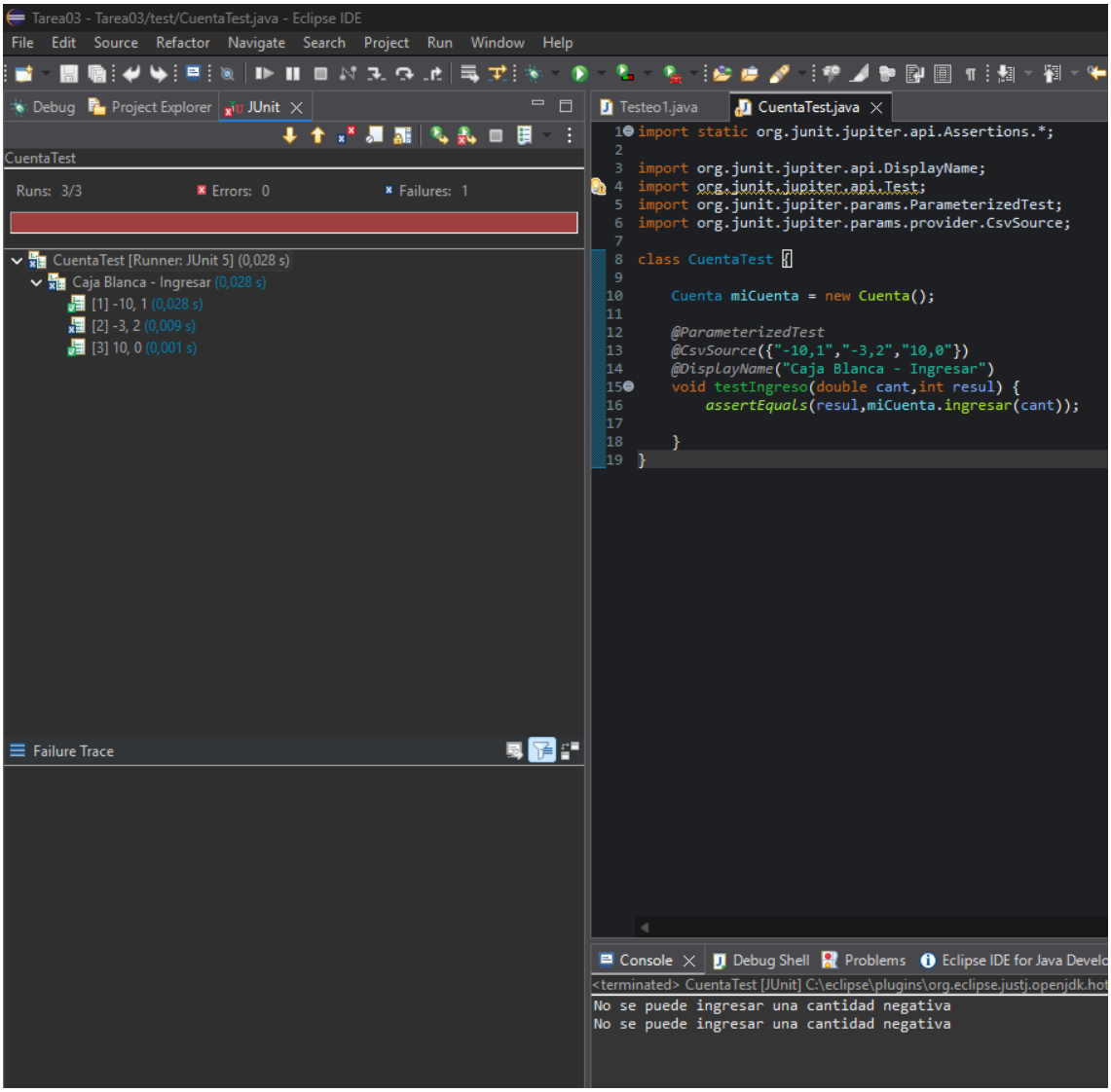
e)delimitar las clases de equivalencia y analizar los valores limites

Condición de entrada	Clases de equivalencia	Clases válidas	COD	Clases no válidas	COD
Cantidad < 0	CE1	X	1	V	1
Cantidad =-3	CE2	X	2	V	2
Cantidad > 0	CE3	V	0	-	-

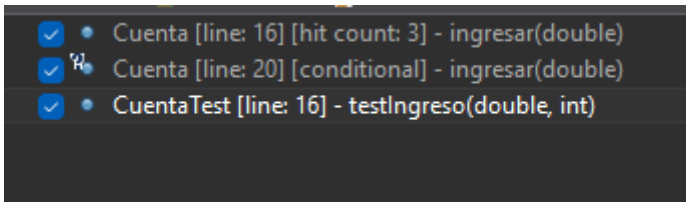
f) casos de uso , resultado esperado

Caso	Entrada	Resultado
1	Cantidad = 0	
2	Cantidad < 0	Error 1
3	Cantidad = -3	Error
4	Cantidad valida con saldo	Retira el dinero

3.Junit5


Tarea03 - Tarea03/test/CuentaTest.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Debug Project Explorer JUnit
CuentaTest
Runs: 3/3 Errors: 0 Failures: 1
CuentaTest [Runner: JUnit 5] (0,028 s)
Caja Blanca - Ingresar (0,028 s)
[1] -10, 1 (0,028 s)
[2] -3, 2 (0,009 s)
[3] 10, 0 (0,001 s)
Failure Trace
Testeo1.java
CuentaTest.java
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
class CuentaTest {
Cuenta miCuenta = new Cuenta();
@ParameterizedTest
@CsvSource({"-10,1","-3,2","10,0"})
@DisplayName("Caja Blanca - Ingresar")
void testIngreso(double cant,int resul) {
assertEquals(resul,miCuenta.ingresar(cant));
}
}
Console Debug Shell Problems Eclipse IDE for Java Develo
<terminated> CuentaTest [JUnit] C:\eclipse\plugins\org.eclipse.justj.openjdk.hot
No se puede ingresar una cantidad negativa
No se puede ingresar una cantidad negativa

4.Puntos de ruptura



1º punto de ruptura

// Depuracion. Punto de parada. Solo en el 3 ingreso

```
14            else{
15                    // Depuracion. Punto de parada.
16                     dSaldo = dSaldo + cantidad;
17                     iCodErr = 0;
```

2º punto de ruptura

```
18            }
19        // Depuracion. Punto de parada cuando la cantidad  es menor
20        return iCodErr;
21
22
```

3º punto de ruptura

```
1 import static org.junit.jupiter.api.Assertions.*;
7
8 class CuentaTest {
9
10     Cuenta miCuenta = new Cuenta();
11
12     @ParameterizedTest
13     @CsvSource({"-10,1","-3,2","10,0"})
14     @DisplayName("Caja Blanca - Ingresar")
15     void testIngreso(double cant,int resul) {
16         assertEquals(resul,miCuenta.ingresar(cant));
17
18     }
19 }
```

5 . Puntos de breakpoint


```
<?xml version="1.0" encoding="UTF-8"?>
<breakpoints>
<breakpoint enabled="true" persistant="true" registered="true">
<resource path="/Tarea03/test/Cuenta.java" type="1"/>
<marker charStart="529" lineNumber="16"
type="org.eclipse.jdt.debug.javaLineBreakpointMarker">
<attrib name="charStart" value="529"/>
<attrib name="org.eclipse.jdt.debug.core.suspendPolicy" value="2"/>
<attrib name="org.eclipse.jdt.debug.ui.JAVA_ELEMENT_HANDLE_ID"
value="=Tarea03/test&lt;{Cuenta.java[Cuenta"/>
<attrib name="org.eclipse.jdt.debug.core.hitCount" value="3"/>
<attrib name="charEnd" value="571"/>
<attrib name="org.eclipse.debug.core.enabled" value="true"/>
<attrib name="org.eclipse.jdt.debug.core.expired" value="false"/>
<attrib name="message" value="Line breakpoint:Cuenta [line: 16] [hit count: 3] -
ingresar(double)"/>
<attrib name="org.eclipse.debug.core.id" value="org.eclipse.jdt.debug"/>
<attrib name="org.eclipse.jdt.debug.core.typeName" value="Cuenta"/>
<attrib name="workingset_name" value=""/>
<attrib name="workingset_id" value="org.eclipse.debug.ui.breakpointWorkingSet"/>
</marker>
</breakpoint>
<breakpoint enabled="true" persistant="true" registered="true">
<resource path="/Tarea03/test/Cuenta.java" type="1"/>
<marker charStart="684" lineNumber="20"
type="org.eclipse.jdt.debug.javaLineBreakpointMarker">
<attrib name="org.eclipse.jdt.debug.core.conditionEnabled" value="true"/>
<attrib name="charStart" value="684"/>
<attrib name="org.eclipse.jdt.debug.core.suspendPolicy" value="2"/>
<attrib name="org.eclipse.jdt.debug.core.condition" value="cantidad &lt; 0"/>
<attrib name="org.eclipse.jdt.debug.ui.JAVA_ELEMENT_HANDLE_ID"
value="=Tarea03/test&lt;{Cuenta.java[Cuenta"/>
```

<attrib name="charEnd" value="705"/>

<attrib name="org.eclipse.debug.core.enabled" value="true"/>

<attrib name="message" value="Line breakpoint:Cuenta [line: 20] [conditional] - ingresar(double)"/>

<attrib name="org.eclipse.debug.core.id" value="org.eclipse.jdt.debug"/>

<attrib name="org.eclipse.jdt.debug.core.typeName" value="Cuenta"/>

<attrib name="workingset_name" value=""/>

<attrib name="workingset_id" value="org.eclipse.debug.ui.breakpointWorkingSet"/>

</marker>

</breakpoint>

<breakpoint enabled="true" persistant="true" registered="true">

<resource path="/Tarea03/test/CuentaTest.java" type="1"/>

<marker charStart="441" lineNumber="16" type="org.eclipse.jdt.debug.javaLineBreakpointMarker">

<attrib name="charStart" value="441"/>

<attrib name="org.eclipse.jdt.debug.core.suspendPolicy" value="2"/>

<attrib name="org.eclipse.jdt.debug.ui.JAVA_ELEMENT_HANDLE_ID" value="=Tarea03/test<{CuentaTest.java[CuentaTest"/>

<attrib name="charEnd" value="487"/>

<attrib name="org.eclipse.debug.core.enabled" value="true"/>

<attrib name="message" value="Line breakpoint:CuentaTest [line: 16] - testIngreso(double, int)"/>

<attrib name="org.eclipse.debug.core.id" value="org.eclipse.jdt.debug"/>

<attrib name="org.eclipse.jdt.debug.core.typeName" value="CuentaTest"/>

<attrib name="workingset_name" value=""/>

<attrib name="workingset_id" value="org.eclipse.debug.ui.breakpointWorkingSet"/>

</marker>

</breakpoint>

</breakpoints>